HongKong Metropolitan University

COMP 8090SEF
Data Structures And Algorithms

Course Project Report

Submission date: 9 April 2026

I declare that:

(i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;

(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;

(iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;

(iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
HE Xue	13927408

Course Project Report
SmartPET - a Pet Shop Management System

1. Introduction

I developed **SmartPET**, an OOP-based pet shop management system that designed to help pet store owners manage their business operations efficiently as my Task 1 of the COMP8090SEF course project. The system provides three core functions: managing pet inventory, tracking customers, and processing sales. It demonstrate Object-Oriented Programming concepts in a practical, real-world scenario.

1.1 Core Functions

Function	Description
Managing Pet Inventory	Add, view, and manage pets with details including name, age, price, type etc.
Tracking Customers	Maintain customer records with contact information and purchase history
Processing Sales	Sell pets to customers with inventory updates and revenue tracking

1.2 Technologies Used

The system is built in **Python**. The project follows modular programming principles with code organized into four separate files: **models.py** for data classes, **petshop.py** for the main controller, **utils.py** for helper functions, and **main.py** for the user interface.

The repository link is:
https://github.com/Xuewaonline/HEXue_SID13927408_COMP8090SEF_IndividualProject/tree/main

The video link is:
https://www.youtube.com/watch?v=SWd6C8hPUxs

2. System Design

The system is designed around a central PetShop class that manages all business operations. This controller class maintains collections of Pet objects and Customer objects, providing methods to add, search, and manipulate these objects. The design follows the principle of separation of concerns, with each class having a specific responsibility within the system.

2.1 OOP Integration

The **Pet class** serves as the data model for pets in the system. It encapsulates pet attributes such as ID, name, age, price, breed, type, and sale status using private variables. Access to these attributes is controlled through getter and setter methods, which demonstrates encapsulation. For example, the price setter includes validation to ensure negative prices are rejected, protecting data integrity.
(Appendix & References: Figure 0 Pet class)

Class attributes are used to maintain data shared across all pet instances. The **pet_id_counter** attribute generates unique IDs for each new pet, and **total_pets** tracks the total number of pets created. The **Customer class** maintains customer information

and tracks their purchase history. When a customer buys a pet, the **buy_pet** method marks the pet as sold and adds it to the customer's purchase list. This creates a relationship between customers and their purchased pets.
(Appendix & References: Figure 1 Class)

The **PetShop class** acts as the main controller. It uses composition to manage collections of pets and customers, establishing "has-a" relationships. The class provides methods for all three core functions: adding pets to inventory, adding and finding customers, and processing sales transactions.
(Appendix & References: Figure 2 PetShop class)

2.2 Data Flow

When a user adds a new pet to the system, a Pet object is created with the specified attributes and added to the PetShop's pet list. The pet is assigned a unique ID using the **pet_id_counter**. When a sale is processed, the PetShop locates the pet by ID, verifies the customer exists, marks the pet as sold, updates the customer's purchase history, and records the transaction in the sales history. Revenue is calculated dynamically based on completed sales.

3. Implementation

3.1 Pet Class

The **Pet class** encapsulates all pet-related data. **Getter methods** provide read access to these attributes, while **setter methods** allow controlled modification with validation. The class also implements magic methods: **__str__** for string representation when printing pet objects, and **__eq__** for equality comparison based on pet ID.
(Appendix & References: Figure 3 Magic Methods)

3.2 PetShop Controller

The **PetShop class** implements the three core functions through dedicated methods.
For **inventory management**, methods include add_pet, get_all_pets, get_available_pets, and find_pet_by_id. For **customer tracking**, methods include add_customer, get_all_customers, and find_customer_by_name. For **sales processing**, the sell_pet method handles the complete transaction workflow.

3.3 OOP Concepts used in SmartPET

Concept	Application
Class and Object	Three classes: Pet, Customer, PetShop
Encapsulation	Private attributes with getters / setters
Class Attributes	Shared ID counter and pet counter
Static Methods	Price formatting
Magic Methods	__str__, __eq__, __len__
Modular Programming	Four files by functionality

4. Reflection

Developing **SmartPET** helped me understand how OOP concepts work together in practice. Encapsulation ensures data integrity by controlling access to object attributes. Class attributes allow sharing data across all instances of a class. Overall, this project reinforced the value of object-oriented design for creating organized and maintainable software.

4.1 Strengths

SmartPET demonstrates OOP concepts through a practical application. The three core functions work together cohesively. The modular structure makes the code organized and maintainable.

4.2 Limitations

The current implementation stores all data in memory, which means information is lost when the program exits.

4.3 Further Improvements

For future improvements, I plan to implement data persistence using file storage or a database. Moreover, a graphical interface can be developed, , which can present it more vividly.

Appendix & References:

Figure 0: Pet class

```
def __init__(self, name, age, price, breed, pet_type):
    """Initialize a pet with private attributes"""
    self.__id = Pet.pet_id_counter
    Pet.pet_id_counter += 1
    Pet.total_pets += 1
    self.__name = name
    self.__age = age
    self.__price = price
    self.__breed = breed
    self.__type = pet_type
    self.__is_sold = False

# Getter methods
def get_id(self):  6 usages (5 dynamic)
    return self.__id
def get_name(self):  7 usages (7 dynamic)
    return self.__name
def get_age(self):  1 usage (1 dynamic)
    return self.__age
def get_price(self):  4 usages (4 dynamic)
    return self.__price
def get_breed(self):
    return self.__breed
def get_type(self):  3 usages (3 dynamic)
    return self.__type
def is_sold(self):  5 usages (5 dynamic)
    return self.__is_sold

# Setter methods
def set_name(self, name):
    self.__name = name
def set_age(self, age):
    if age > 0:
        self.__age = age
def set_price(self, price):
    if price >= 0:
        self.__price = price
def mark_as_sold(self):  1 usage (1 dynamic)
    self.__is_sold = True
```

Figure 1: Class

```
class Pet:  6 usages
    # Class attributes - shared by all Pet objects
    total_pets = 0
    pet_id_counter = 100

    def __init__(self, name, age, price, breed, pet_type):

        self.__id = Pet.pet_id_counter
        Pet.pet_id_counter += 1
        Pet.total_pets += 1
        self.__name = name
        self.__age = age
        self.__price = price
        self.__breed = breed
        self.__type = pet_type
        self.__is_sold = False
```

Figure 2: PetShop class

```python
class PetShop:  5 usages

    shop_name = "SmartPET"

    def __init__(self, owner_name):
        """Create a new pet shop"""
        self.__owner = owner_name
        self.__pets = []
        self.__customers = []
        self.__sales_history = []
        self.__total_revenue = 0

    # Getter methods
    def get_owner(self):  2 usages (1 dynamic)
        return self.__owner

    def get_total_revenue(self):  2 usages (2 dynamic)
        return self.__total_revenue

    def get_pet_count(self):  2 usages (1 dynamic)
        return len(self.__pets)

    def get_customer_count(self):  1 usage (1 dynamic)
        return len(self.__customers)
```

Figure 3: Magic Methods

```python
# Magic method - string representation
def __str__(self):
    status = "Sold" if self.__is_sold else "Available"
    return "[{}] {} | {} | {} years | ${} | [{}]".format(
        *args: self.__id, self.__name, self.__type,
        self.__age, self.__price, status
    )

# Magic method - equality comparison
def __eq__(self, other):
    if isinstance(other, Pet):
        return self.__id == other.get_id()
    return False
```

References:

Course Lecture Notes: Object-Oriented Programming_lec_note_2_OOP(2026)

Python 3 - Episode 33 - Pet Shop Application
https://www.youtube.com/watch?v=UQslB5ElFE4

Object-Oriented Programming *https://kblog.ink/?p=302*